Side Activity 10: Correlated Subqueries & Set Operations

CSCI3170 Introduction to Database Systems

Learning Objectives

In this Side Activity, you will learn:

- How **correlated subqueries** differ from the non-correlated subqueries you wrote in Side Activity 8.
- How to place a correlated subquery in `WHERE`, `SELECT`, and `HAVING` clauses.
- How to use correlated **EXISTS** and **NOT EXISTS** to test for the presence or absence of related rows.
- How to combine query results using **set operations**: `UNION`, `INTERSECT`, and `MINUS`.

Part 0: Getting Started

We continue with the same **University schema** from Side Activitys 8 and 9. If your tables are still in place, run a quick sanity check:

```
SELECT COUNT(*) AS num_students   FROM student;
SELECT COUNT(*) AS num_courses    FROM course;
SELECT COUNT(*) AS num_sections   FROM section;
SELECT COUNT(*) AS num_takes_rows FROM takes;
```

Expected: 13 students, 13 courses, 15 sections, 22 takes rows. If your numbers differ, re-run:

```
@DDL.sql
@smallRelationsInsertFile.sql
```

Recap: Side Activity 8 Subquery Types

In Side Activity 8, we learned three subquery patterns. All of them were **non-correlated** — the inner query runs once and produces a fixed result:

- **Scalar subquery**: returns one value (e.g., `SELECT MAX(salary) FROM instructor`).
- **IN / NOT IN**: returns a set for membership testing.
- **Inline view**: a subquery in the `FROM` clause used as a temporary table.

Today we extend this to **correlated** subqueries, where the inner query depends on the current row of the outer query.

Part 1: Correlated Subqueries

Key Concept: Correlated vs. Non-correlated

A **non-correlated** subquery (Side Activity 8) can run independently — it is evaluated **once** and the result is reused for every outer row.
A **correlated** subquery references a column from the outer query. Conceptually, it is re-evaluated **for each row** of the outer query, because the referenced value changes from row to row.

Demo 1: Instructors earning above their department average

Recall Side Activity 8, where we found the instructor with the globally highest salary using a non-correlated scalar subquery:

```
-- Non-correlated (Side Activity 8 style): compare against one global value
SELECT i.ID, i.name, i.salary
FROM instructor i
WHERE i.salary = (SELECT MAX(salary) FROM instructor);
```

The subquery `SELECT MAX(salary) FROM instructor` runs once and returns a single number. Every outer row is compared against that same number.

Now suppose we want a different question: find instructors whose salary exceeds the average **of their own department**:

```
-- Correlated: the subquery references i.dept_name from the outer row
SELECT i.ID, i.name, i.dept_name, i.salary
FROM instructor i
WHERE i.salary > (
  SELECT AVG(i2.salary)
  FROM instructor i2
  WHERE i2.dept_name = i.dept_name    -- correlation
)
ORDER BY i.dept_name, i.salary DESC;
```

The key difference is `WHERE i2.dept_name = i.dept_name` inside the subquery — this makes the average **different for each outer row**. When the outer row is a Physics instructor, the subquery computes the Physics average; when it moves to a Comp. Sci. instructor, it computes the Comp. Sci. average.

How to identify a correlated subquery

Look inside the subquery for a reference to a table alias defined in the **outer** `FROM` clause. In Demo 1, `i.dept_name` inside the subquery refers to the outer table alias `i`. That reference is the "correlation".

Demo 2: Correlated subquery in the SELECT list

You can also place a correlated subquery in the `SELECT` clause to compute a per-row value. For example, show each department name together with how many instructors it has:

```
SELECT d.dept_name,
       (SELECT COUNT(*)
        FROM instructor i
```

```
        WHERE i.dept_name = d.dept_name) AS num_instructors
FROM department d
ORDER BY num_instructors DESC, d.dept_name;
```

For each row in `department`, Oracle evaluates the inner `COUNT(*)` using only the instructors in that department.

SELECT-list subquery must return exactly one value

A subquery in the `SELECT` list must be a **scalar** subquery (one row, one column). If it could return more than one row, Oracle raises an error. Aggregate functions like `COUNT`, `AVG`, `MAX` are safe because they always produce a single value.

Demo 3: Correlated subquery with NVL

Find departments whose total budget is larger than the sum of all instructor salaries in that department:

```
SELECT d.dept_name, d.budget
FROM department d
WHERE d.budget > (
  SELECT NVL(SUM(i.salary), 0)
  FROM instructor i
  WHERE i.dept_name = d.dept_name
)
ORDER BY d.dept_name;
```

`NVL(SUM(i.salary), 0)` handles the edge case where a department has no instructors (the `SUM` would return NULL).

Part 2: Correlated EXISTS / NOT EXISTS

Key Concept: EXISTS is inherently correlated

`EXISTS (subquery)` returns TRUE if the subquery produces **at least one row**. In practice, the subquery almost always references the outer query (otherwise it would be TRUE for every outer row or FALSE for every outer row, which is rarely useful).
Think of `EXISTS` as asking: "For this particular outer row, can I find at least one matching inner row?"

Demo 4: EXISTS — students who have taken CS-101

In Side Activity 8, we solved a similar question with `IN`. Here is the `EXISTS` version — compare the two styles:

```
-- Side Activity 8 style (IN, non-correlated):
SELECT s.ID, s.name
FROM student s
WHERE s.ID IN (
  SELECT t.ID FROM takes t WHERE t.course_id = 'CS-101'
);

-- EXISTS (correlated):
SELECT s.ID, s.name
```

```
FROM student s
WHERE EXISTS (
  SELECT 1
  FROM takes t
  WHERE t.ID = s.ID              -- correlation
    AND t.course_id = 'CS-101'
)
ORDER BY s.ID;
```

Both return the same result. The difference in approach:

- `IN`: the subquery runs once, builds a set of IDs, and the outer query checks membership.
- `EXISTS`: for each outer student row, the subquery checks whether a matching `takes` row exists.

Demo 5: NOT EXISTS — instructors who have never taught

`NOT EXISTS` keeps an outer row only when the correlated subquery returns **no rows**. This is the "anti-join" pattern: find rows in one table that have **no match** in another.

```
SELECT i.ID, i.name, i.dept_name
FROM instructor i
WHERE NOT EXISTS (
  SELECT 1
  FROM teaches t
  WHERE t.ID = i.ID              -- correlation
)
ORDER BY i.dept_name, i.ID;
```

How to read this query: "For each instructor `i`, check the `teaches` table. If there is **no row** where `t.ID = i.ID`, then this instructor has never taught — keep them in the result."

Demo 6: NOT EXISTS — courses with no prerequisites

Another anti-join example. Find courses that do not appear in the `prereq` table:

```
SELECT c.course_id, c.title
FROM course c
WHERE NOT EXISTS (
  SELECT 1
  FROM prereq p
  WHERE p.course_id = c.course_id
)
ORDER BY c.course_id;
```

Compare this with the `NOT IN` version from Side Activity 8 — both produce the same result, but `NOT EXISTS` is NULL-safe (see warning below).

NOT EXISTS vs NOT IN: NULL safety

`NOT EXISTS` is **NULL-safe**: it simply checks whether any row is returned.
`NOT IN` can silently return **no rows at all** if the subquery result contains a NULL value (because any comparison with NULL yields UNKNOWN, and the entire `NOT IN` condition becomes UNKNOWN).
Rule of thumb: when writing anti-join queries ("find rows with no match"), prefer `NOT EXISTS` to avoid NULL surprises.

Part 3: Set Operations

Key Concept: UNION, INTERSECT, MINUS

Set operations combine the **result sets** of two `SELECT` queries:

- `UNION`: all rows from either query (duplicates removed).
- `UNION ALL`: all rows from either query (duplicates **kept**).
- `INTERSECT`: only rows that appear in **both** results.
- `MINUS`: rows in the first result that are **not** in the second.

Requirements: both queries must have the **same number of columns** with compatible data types. Column names in the output come from the **first** query.
Oracle note: Oracle uses `MINUS` where the SQL standard uses `EXCEPT`.

Demo 7: UNION — all people in the Comp. Sci. department

Students and instructors live in separate tables. To combine them into one list:

```
SELECT name FROM student     WHERE dept_name = 'Comp. Sci.'
UNION
SELECT name FROM instructor  WHERE dept_name = 'Comp. Sci.'
ORDER BY name;
```

If a person is both a student and an instructor with the same name, `UNION` keeps only one copy.

UNION vs UNION ALL

`UNION` removes duplicates (like applying `DISTINCT` to the combined result). `UNION ALL` keeps every row, including duplicates.
Use `UNION ALL` when you **know** there are no duplicates or when you **want** to keep them — it is faster because it skips the deduplication step.

Demo 8: MINUS — courses offered in Fall 2017 but not in Spring 2018

`MINUS` returns rows from the first query that do not appear in the second query:

```
SELECT course_id FROM section WHERE semester = 'Fall'   AND year = 2017
MINUS
SELECT course_id FROM section WHERE semester = 'Spring' AND year = 2018
ORDER BY course_id;
```

This is equivalent to using `NOT IN` or `NOT EXISTS`, but often more readable for simple "set A minus set B" problems.

Demo 9: INTERSECT — courses offered in both terms

`INTERSECT` returns only rows that appear in both query results:

```
SELECT course_id FROM section WHERE semester = 'Fall'   AND year = 2017
INTERSECT
SELECT course_id FROM section WHERE semester = 'Spring' AND year = 2018
ORDER BY course_id;
```

When to use which?

Many problems can be solved with either subqueries or set operations. A quick guide:

- **Set operations** are cleanest when both sides have the same columns and you want a simple union/difference/intersection of two sets.
- **Correlated subqueries** (EXISTS/NOT EXISTS) are more flexible when the two sides have different structures or when you need to correlate on multiple columns.

For example, "instructors who never taught" can be solved with either `MINUS` or `NOT EXISTS` — try both and see which you find more readable.

Part 4: Self Exercises

All questions use the **University schema**.

Correlated subquery questions

Q1. (Correlated subquery in WHERE) Find all instructors whose salary is strictly above the average salary of their own department. Output: `ID, name, dept_name, salary`. Order by `dept_name`, then `salary` descending.

Q2. (Correlated subquery in SELECT) For each department, display the department name and the number of distinct students who have taken at least one course offered by that department. Use a correlated subquery in the `SELECT` list. Output: `dept_name, student_count`. Order by `student_count` descending.

Hint: The path is `department` → `course` → `takes`.

EXISTS / NOT EXISTS questions

Q3. (EXISTS) Find all courses that have been offered at least once (i.e., there exists a row in `section` for that course). Use `EXISTS`. Output: `course_id, title`. Order by `course_id`.

Q4. (NOT EXISTS) Find all students who have never taken any course from the **Comp. Sci.** department. Use `NOT EXISTS`. Output: `ID, name, dept_name`. Order by `ID`.

Hint: The subquery should join `takes` with `course` and check `course.dept_name`.

Set operation questions

Q5. (UNION) Produce a single list of names of all people (students and instructors) who belong to the **Comp. Sci.** department. Remove duplicates. Order by `name`.

Q6. (MINUS) Find the IDs of all instructors who have never taught a course. Use `MINUS` (Oracle's equivalent of `EXCEPT`). Order by `ID`.

Note: Compare your result with the `NOT EXISTS` approach in Demo 5.

Challenge

Q7. (Challenge: double NOT EXISTS) Find all students who have taken **every** course offered by the Comp. Sci. department. Output: `ID, name`. Order by `ID`.

Hint: Read the condition as: "there is **no** CS course such that this student has **not** taken it." Translate each "no" into a `NOT EXISTS`.

Note: The correct answer may be an empty result with this dataset.

End of Side Activity 10.